

PYTHON (BEGINNING TO INTERMEDIATE)

Let's go through Python from the basics to intermediate concepts, explaining everything in simple and detailed language. Here's the breakdown:

Python doesn't need to be compiled into machine code first (like C or C++). Instead, Python code is executed line by line by an interpreter.

1. History of Python:

Python is a high-level, interpreted programming language created by **Guido van Rossum** in the late 1980s and released in **1991**. Van Rossum was influenced by the ABC programming language and wanted to create a language that was easy to read, with an emphasis on simplicity and code readability.

Python was named after a British comedy group called "Monty Python's Flying Circus."

Python has evolved over the years, with major versions like:

- **Python 2.x** (released in 2000): This version had widespread use, but was eventually discontinued in **2020**.
- **Python 3.x** (released in 2008): This is the current version and includes many improvements over Python 2. Python 3 is not backward compatible with Python 2, meaning Python 2 code doesn't always work on Python 3.

Python is known for its clean syntax, making it easier to learn and write code.

2. Basic Python Concepts:

Variables:

Variables in Python are used to store data values. You don't need to declare the type of the variable, Python automatically determines it based on the assigned value.

Example:

```
name = "John" # A string
```

```
age = 25      # An integer
```

Data Types:

Python has several built-in data types:

- **Numbers:** Integers (int) and floating-point numbers (float).
- **Strings:** A sequence of characters enclosed in single, double, or triple quotes.
- **Booleans:** Used for representing truth values (True or False).
- **None:** A special type representing "nothing" or "no value".

Example:

```
x = 10      # Integer
```

```
y = 3.14    # Float
```

```
z = "Hello" # String
```

```
flag = True  # Boolean
```

```
n = None    # None
```

Operators:

Operators perform operations on variables and values.

Common types are:

- **Arithmetic Operators:** +, -, *, /, // (floor division), % (modulo), ** (exponentiation).
- **Comparison Operators:** ==, !=, >, <, >=, <=.
- **Logical Operators:** and, or, not.
- **Assignment Operators:** =, +=, -=, *=, /=, etc.

Example:

```
a = 10
```

```
b = 5
```

```
c = a + b # c is 15
```

3. Control Flow:

Conditionals:

You can control the flow of your program using if, elif, and else statements.

Example:

```
age = 20
```

```
if age >= 18:
```

```
    print("Adult")
```

```
else:
```

```
    print("Minor")
```

Loops:

Python has two types of loops:

- **For loop:** It iterates over a sequence (like a list, string, or range).
- **While loop:** It keeps executing as long as the condition is True.

Example of a for loop:

```
for i in range(5): # Iterates from 0 to 4
    print(i)
```

Example of a while loop:

```
count = 0
while count < 5:
    print(count)
    count += 1
```

4. Functions:

A function is a block of code that only runs when it is called.
Functions can take parameters and return values.

Example:

```
def greet(name):
    return f'Hello, {name}'
```

```
message = greet("Alice")
```

Parameter- A variable in the function definition (like a placeholder).
Argument- The actual value you pass to the function when calling it.

```
print(message)
```

- **Arguments:** The data you pass into a function.
 - **Return:** The value the function gives back when it's done executing.
-

5. Collections:

Lists:

Lists are ordered collections that can hold multiple data types.

You can modify them (they are mutable).

Example:

```
fruits = ["apple", "banana", "cherry"]
```

```
fruits.append("orange") # Adds an item
```

Tuples:

Tuples are similar to lists, but they are **immutable** (can't be changed after creation). They are often used to represent fixed collections of related data.

Example:

```
coordinates = (10, 20) # A tuple with two elements
```

Key Points:

- Tuples are often used when you need to ensure that data doesn't change.

- They are created using parentheses () instead of square brackets [].
-

6. Dictionaries: ** were unordered only till py vers. <3.7. 3.7 onwards theyre ordered

Dictionaries are unordered collections that store data in key-value pairs. The key is unique, and you use the key to access its associated value.

Example:

```
person = {"name": "Alice", "age": 25}
print(person["name"]) # Accessing the value using the key
```

Key Points:

- Dictionaries are mutable.
 - You can add, remove, or change the key-value pairs.
-

7. Intermediate Concepts:

List Comprehensions:

List comprehensions provide a concise way to create lists. Instead of writing a full for loop, you can generate a list in a single line.

Example:

```
squares = [x**2 for x in range(5)] # [0, 1, 4, 9, 16]
```

Lambda Functions:

A lambda function is a **small anonymous function defined using the lambda keyword**. It can take **any number of arguments** but only has one expression.

Example:

```
add = lambda x, y: x + y  
print(add(2, 3)) # Output: 5
```

File Handling:

You can work with files using Python's built-in functions. The `open()` function is used to open a file, and you can read/write to it.

Example:

```
# Opening a file in write mode  
with open("file.txt", "w") as f:  
    f.write("Hello, world!")
```

Error Handling (Exceptions):

Python **uses try, except, and finally blocks to handle errors**. This allows you to **manage unexpected situations and prevent program crashes**.

Example:

```
try:  
    result = 10 / 0  
except ZeroDivisionError:  
    print("Cannot divide by zero!")
```

finally:

```
print("Execution finished.")
```

8. Object-Oriented Programming (OOP):

Python supports OOP, where you organize code into classes and objects.

- **Class:** A blueprint for creating objects.
- **Object:** An instance of a class.

Example:

```
class Dog:
```

```
    def __init__(self, name, age):
```

```
        self.name = name
```

```
        self.age = age
```

```
    def bark(self):
```

```
        return f'{self.name} says Woof!'
```

```
dog1 = Dog("Buddy", 3)
```

```
print(dog1.bark())
```

Key Concepts in OOP:

- **Encapsulation:** Bundling data and methods that operate on the data.

- **Inheritance:** A way for one class to inherit properties and methods from another class.
 - **Polymorphism:** The ability to use the same method name but perform different tasks depending on the object.
-

9. Modules and Libraries:

Python has a rich set of libraries and modules. You can use them by importing them into your script.

Example:

```
import math
```

```
print(math.sqrt(16)) # Output: 4.0
```

Modules are a way to organize code into separate files. The Python Standard Library is packed with useful modules like `math`, `os`, `sys`, and `datetime`.

This covers the basics and intermediate concepts in Python. Let me know if you'd like to dive deeper into any topic or need further explanations!